Revision — 1.2.3 Software Development

OCR H446 — one-page revision summary

Must know

- A **development methodology** is the overall approach used to plan, build and manage a software project. Choose it to fit the scenario (clarity of requirements, budget, deadline, risk, team).
- **Waterfall** is **linear/sequential** — each stage is fully completed before the next. Simple and well documented, but **inflexible** and the client only sees the product at the **end**. Use when requirements are **fixed and clear**.
- **Agile** delivers **iterative increments** with **continuous client feedback**, so changing requirements are built into later cycles. Flexible but with lighter documentation and less predictable cost/time. Use when requirements are **unclear or likely to change**.
- **Extreme programming (XP)** is Agile plus **pair programming**, constant/test-driven testing and a customer in the team — very high code quality but staff-intensive.
- **Spiral** is iterative with **risk analysis** at the centre of each loop — best for **large, expensive, high-risk** projects; risks are found early, but it is complex and costly.
- **RAD (Rapid Application Development)** relies on **prototyping** refined by user feedback — fast delivery for vague requirements, but needs skilled developers and committed users and can be less robust.
- **Lifecycle stages (in order): Analysis** (investigate the problem, gather requirements, assess feasibility) → **Design** (data structures, algorithms, user interface, inputs/outputs/file formats) → **Development/implementation** (code in modules) → **Testing** (against requirements) → **Evaluation**.
- **Testing** uses three categories of data: **normal/valid** (typical, in-range, should be accepted), **boundary** (values at the **edges** of the acceptable range), and **erroneous/invalid** (outside the range, should be rejected). Also know **alpha** (in-house) and **beta** (real users) testing.
- **Evaluation** judges the finished system against the **original requirements** — is it effective, usable, robust and maintainable?

Key terms

Term	Definition
Waterfall	Linear, sequential methodology completing one stage before the next
Agile	Iterative increments with continuous client feedback
Extreme programming (XP)	Agile with pair programming and constant/test-driven testing
Spiral	Iterative methodology with risk analysis at the centre of each loop
RAD	Rapid prototyping refined by user feedback
Boundary data	Test values at the edges of the acceptable range
Erroneous data	Invalid test values outside the acceptable range, to be rejected
Evaluation	Judging the finished system against the original requirements

Worked example / how to — matching a methodology to a scenario

Scenario: a new mobile app; requirements are unclear and expected to change as users see early versions; modest budget; small, skilled team. - Recommend **Agile** (or RAD): its **iterative increments** and **continuous user feedback** let evolving ideas be added each cycle rather than fixed up front. - The **small, skilled team** copes with Agile's lighter documentation; the **modest budget/short timescale** suits it better than heavily documented Waterfall, which would lock requirements too early. - *Justify against the scenario clues* — pure description caps the marks.

Exam tips

- Methodology questions are **AO2/AO3**: apply and **justify** your choice using scenario clues (budget, deadline, clarity of requirements, risk, team size).
- Learn the lifecycle stages **in order** and one or two concrete tasks for each.
- Name all **three test-data categories** and give a clear example of each — boundary means the **edge** value, not "just outside".
- For Spiral, always link **risk analysis each loop** to catching costly problems early on **high-risk/high-cost** projects.